© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

Duración: 2h15m **Nombre:**

1. (1.0 ptos) Dadas las siguientes afirmaciones, indica si son verdaderas (V) o falsas (F). Las contestaciones erróneas restan la mitad de la puntuación.

a) $n\,lg\,n \in O(n\sqrt{n})$ [ ]

b) $\sqrt{n}\,lg\,n \in \Omega(n)$ [ ]

c) $20n\,lg\,n^2 + 3n \in O(n^{1.2})$ [ ]

d) $\Theta(5n^2 + 2n + 3) = \Theta(n^2)$ [ ]

e) $\lg\big(\lg\big(2^{\sqrt{n}}\big)\big) \in \Omega(\lg n)$ [ ]

2. (1.5 ptos)

```
func f(a: vector[1..n] de Natural) dev s: Natural
s ← 0
para i ← 2 hasta n-1
      si a[i] > 0
            para j ← 1 hasta i+1
                  s ← s + a[j]
            fpara
      fsi
fpara
```

a) Número de instancias diferentes.

b) Calcula el coste exacto en pasos de los casos mejor y peor.

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

c) Coste asintótico de la función.

3. (1.0 ptos) Completa la siguiente función recursiva. Ten en cuenta que es posible llamarla con un vector vacío (N = 0).

```
func f(v: Vector[1..N] de Real; i: Natural; x: Real) dev res: Bool

si Cond
    res ← E1
sino
    res ← f(v, E2, 2*x)
    res ← E3

{Post: res = Σα: 1 ≤ α ≤ i: (v[N-α+1]*x) }
```

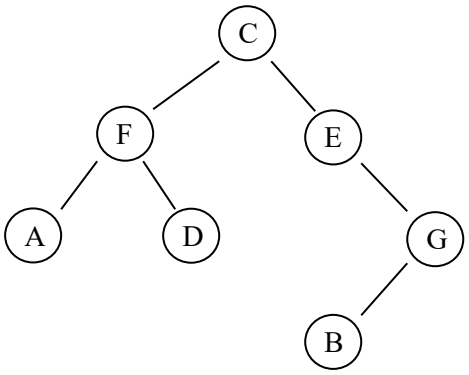
Cond:
E1:
E2:
E3:

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

4. (1.25 ptos) Calcula el **coste exacto** en pasos y el **coste asintótico** de la siguiente función recursiva. Se puede suponer que $n=2^k$ siendo k natural. (Resolver en última página)

```
func f(n: N) dev s: N
si n < 1
      s ← 0
sino
      s ← 0
      a ← f(n/2)
      para i ← 1 hasta n
            s ← s + a
      fpara
fsi
```

5. (1.0 ptos) Dado el siguiente árbol binario:



Recorre el árbol en orden **prefijo** y muestra los nodos en el orden en que se procesan.

a) Recorre el árbol en orden **infijo** y muestra los nodos en el orden en que se procesan.

b) Inserta los siguientes valores en un **Árbol Binario de Búsqueda**: 5,7,3,6,1,9,8.

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

6. (1.0 pto) El siguiente vector representa un montículo de mínimos.

	1	2	3	4	5	6	7	8	9
V:	2	6	4	7	8	5	10	12	9

a) Dibuja el montículo en forma de árbol binario.

b) Inserta el valor 3 en el montículo y muestra el resultado final.

c) Elimina el mínimo del **montículo original** y muestra el resultado final.

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

7. (1.0 ptos) Escribe un algoritmo de búsqueda en profundidad en C++ para un grafo implementado mediante matriz de adyacencia. Se considera que existe un arco si el peso del arco es mayor que 0. El procesado de los nodos será simplemente mostrarlos por pantalla. La cabecera de la función será la siguiente:

```
void Grafo::dfsRec(unsigned origen, vector<bool> & visit)
```

Y la función auxiliar para llamar a ésta será:

```
void Grafo::dfs(unsigned origen)
{
    vector<bool> visit(size(), false); // Cjto nodos visitados
    dfsRec(origen, visit);
}
```

Clase Grafo:

```
class Grafo
{
public:
    typedef ?? TipoPeso;

    Grafo(unsigned);
    TipoPeso & operator()(unsigned f, unsigned c);
    const TipoPeso & operator()(unsigned f, unsigned c) const;
    unsigned size() const;
private:
    vector<vector<TipoPeso> > matriz;
};
```

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

8. (1.0 ptos) Dado el algoritmo de Prim para matriz de adyacencia, del cual se muestra el comienzo a continuación, indicar qué valores contendrán los vectores *vecino* y *pvecino* después de ejecutarlo sobre el grafo representado por la matriz *p*. Indica su valor después de la inicialización y después de cada iteración del bucle principal.

```
ALGORITMO PRIM    // Matriz de adyacencia //
DATOS:       G = (N,A,p) // Grafo no dirigido con pesos. p Matriz de adyacencia
RESULTADO:   vecino: vector[1..n] de 1..n //Mejores arcos factibles
AUX:
             pvecino: vector[1..n] de R    // … y sus pesos.
METODO:
// Inicialización
pvecino[1] ← -1
Para i2 ← 2 hasta n hacer
  vecino[i2] ← 1
  pvecino[i2] ← p[i2,1]
fPara
// Bucle principal
Para i ← 2 hasta n hacer
...
fPara
```

Matriz *p*: El guión (-) representa que no hay arco, es decir, valor infinito.

	1	2	3	4	5
1	0	8	11	3	-
2	8	0	4	1	-
3	11	4	0	10	7
4	3	1	10	0	5
5	-	-	7	5	0

Inicialización

	1	2	3	4	5
vecino:					
pvecino:					

i=2:

	1	2	3	4	5
vecino:					
pvecino:					

i=3:

	1	2	3	4	5
vecino:					
pvecino:					

i=4:

	1	2	3	4	5
vecino:					
pvecino:					

i=5:

	1	2	3	4	5
vecino:					
pvecino:					

Indica el peso total del árbol resultante:

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

9. (1.25 ptos) Dado el siguiente problema:

"*Tenemos n objetos cuyos pesos en gramos vienen dados por el vector p[1..n] de Reales. Tenemos además una balanza en cuya parte derecha hemos puesto una pesa de X gramos. ¿Es posible equilibrar la balanza utilizando una selección de los objetos que tenemos?*"

Escribe un algoritmo que resuelva este problema, es decir, que diga si es posible equilibrar la balanza o no (*igual*), y que además devuelva con que objetos se equilibraría (*c*).

```
ALGORITMO Suma (i:Nat; total: Real)
DATOS:     p: vector[1..n] de Real;  // Pesos
RES/GLOBAL:      c: vector[1..n] de Bool  // Vector resultado
           igual: Bool;     // Equilibrio
INICIO: igual ← falso   // primera llamada Suma(0,X)
METODO:
Si i=n entonces
```


```
sino
  si [                                ] entonces
```


```
    Suma( [                          ] )
  fSi
  si [                                ]
```


```
    Suma( [                          ] )
  fSi
fSi
```